S-boxのΔ-rankとΓ-rankについて（I）

松井 充[1,a)]

概要：本稿ではS-boxのCCZ不変量として知られるΔ-rankとΓ-rankの計算手法について考察を行う．Δ-rankとΓ-rankはS-boxのCCZ非同値性の証明手段とともにS-boxの分類手段としても有効であるが，これらの値を求めるためにはサイズ2^{2n}のバイナリ正方行列のランク計算が必要があり，このための計算時間とメモリ量はS-boxのサイズとともに急速に増加する．本稿ではΔ-rankとΓ-rankの値は一般にランダム行列のランクよりはるかに小さいことを指摘し，このことを用いてメモリ量を削減したΔ-rankとΓ-rankの計算手法を提案するとともに，Δ-rankをPC上で$n = 11$まで求めた初めての例を示す．

キーワード：共通鍵暗号, S-box, APN関数, CCZ同値

On Δ-rank and Γ-rank of S-boxes (I)

Mitsuru Matsui[1,a)]

Abstract: This paper examines methods for computing Δ-rank and Γ-rank, which are known as CCZ invariants of S-boxes. While Δ-rank and Γ-rank are effective tools for proving CCZ-inequivalence and classifying S-boxes, calculating these values requires determining the rank of a binary square matrix of size 2^{2n}; consequently, as the S-box size increases, the memory and computational time grow rapidly. We point out that the values of Δ-rank and Γ-rank are generally much smaller than the rank of a random matrix. Leveraging this observation, we propose a computational method that reduces memory usage and present the first instance of computing the Δ-rank of an S-box for $n = 11$ on a personal computer.

Keywords: Symmetric Key Cipher, S-box, APN function, CCZ equivalence

1. はじめに

多くの共通鍵暗号の構成要素である非線形関数S-boxの安全性評価は，共通鍵暗号の主要な研究テーマであり続けている．特に差分解読法[1]や線形解読法[2]に対する評価は現在も活発に研究されており，しかも数多くの未解決問題が残されている分野でもある．

差分解読法に対するS-boxの安全性を最初に定式化したのはNybergである．Nybergはその一連の研究の中でKnudsenとともに差分解読法に対して最も強い性質を持つS-boxをAPN (almost perfect nonlinear)関数と定義した[3]．またChabaudとVaudenayは線形解読法に対して最も強い性質をもつS-boxをAB (almost bent)関数と名付けた[4]．

これらの性質はある種の双対性を持っており，線形解読法に対して最も強いS-boxすなわちAB関数は，差分解読法に対しても最も強いS-boxすなわちAPN関数でもある（逆は成り立たない）ことを示すことができる[4]．すなわち差分解読法に強いS-boxの性質を調べることは線形解読法に強いS-boxを調べることとも密接に関係している．

APN関数に関する最大の未解決問題は“The Big Open Problem”と呼ばれる「偶数ビットの全単射なAPN関数は存在するか？」という問いであり，そのような例は2009年にビット数$n = 6$の場合にひとつ発見された[5]ものの，それ以外に存在するかどうかは知られていない．またAB関数はその定義からnが奇数の場合にしか存在し得ない

[1] 情報通信研究機構 サイバーセキュリティ研究所
Cybersecurity Research Institute, National Institute of Information and Communications Technology
[a)] linear@nict.go.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

が，偶数の場合にどこまで AB に近い関数が存在するかも未解決である．

APN 関数は最近になって研究が進み，代数次数が 2 である場合に限れば計算機探索によって多数の APN 関数が発見されており，特に応用上重要な $n = 8$ の場合には数百万個の APN 関数が報告されている [6] が，その中に全単射なものは含まれていない．一方で，代数次数が 2 を超える（正確には CCZ 同値類に代数次数 2 の関数を含まない）APN 関数については，昔から知られているいくつかのべき乗関数を除けば筆者が知る限り $n = 6$ の場合の 1 例 [7] しか見つかっていない．

ここで APN 関数探索において「新しい APN 関数」とは何かに注意する必要がある．新しいとは既知の APN 関数のいずれとも同値ではないことを意味するが，この同値関係にはいくつかの種類がある．APN の性質を保つ同値関係のうちその同値類が最も大きいものは CCZ 同値と呼ばれるものであり [8]，ある APN 関数を新しいと主張するためには既知のすべての APN 関数と CCZ 同値ではないことを示さなければならない．

CCZ 同値判定問題は符号同値判定問題に帰着できるため，汎用的な CCZ 同値判定目的には符号同値判定アルゴリズムが用いられるが，符号同値判定問題は少なくともグラフ同値判定問題以上の難しさであることが知られており [9]，既知の多数の関数との非同値性を判定するのは一般には計算時間が膨大になる．そのため実際には，CCZ 同値である限り値が変わらないような S-box の不変量 —但し値が一致しても CCZ 同値とは限らないことを許容する— を CCZ 非同値性の判定に用いることも少なくない．

このような CCZ 不変量としては例えば extended Walsh spectrum がよく知られているが，それよりも解像度が高い，すなわち値の範囲が広く，CCZ 非同値な S-box をより多く捕捉できると考えられているものが Edel と Pott によって提案された Δ-rank と Γ-rank である [7]．これらは与えられた S-box から定義される群代数（group algebra）のある単項イデアルの次元として定義され，その値はあるデザイン（design）の接続行列（incidence matrix）のランクを計算することで得られる．

Δ-rank と Γ-rank は S-box の類別や存在証明の手段としても用いることもできる．例えば，CCZ 同値は非常に広い同値関係であり全単射性も代数次数も保存しないため，同じ同値類の中に異なる代数次数の関数が存在することがある．一方で代数次数が 2 となる APN 関数の Δ-rank はある一定値以下になることが証明できるので，[7] ではこのことを用いて，代数次数が 2 となるいかなる APN 関数とも CCZ 同値ではない APN 関数の存在を，その関数の Δ-rank の値だけから示すことに成功している．

本稿では Δ-rank と Γ-rank をとりあげ，これらの値の計算手法について議論する．S-box のビット数 n に対してその接続行列はサイズ 2^{2n} のバイナリ正方行列になるため，ランク計算に必要な時間やメモリ量は一般に大きなものになる．Kaleyski の論文 [10] には 500GB のメモリを持つサーバー計算機で Δ-rank は $n = 9$ まで，Γ-rank は $n = 10$ までの計算時間が示されているが，それ以上の n ではメモリ量がボトルネックになるとある．また最近の論文 [11] では，より多くのメモリを搭載した計算機での $n = 9$ までの Δ-rank と Γ-rank の計算時間とメモリ量が示されているが，計算時間は [10] よりはるかに大きい．

これらの論文で具体的にどのような関数を対象としたのか不明であるが，まず本稿では，オープンソースライブラリ M4RI を用いて Δ-rank と Γ-rank を求めるプログラムを実装した結果，その計算時間は n が同じであっても関数によって大きく差が出ることを指摘するとともに，より高速かつ少ないメモリ量で計算できたことを報告する．

続いて，S-box の接続行列は一般に同サイズのランダムな行列よりはるかにランクが小さく，最大でも行列サイズの半分にしかならないことを示し，このことを用いた Δ-rank と Γ-rank の新たな計算アルゴリズムを提案する．これは接続行列の生成とランク計算をインターリーブさせるもので，特にランク落ちが大きい Δ-rank の計算では必要なメモリ量を大幅に削減するものである．

最後に，このアルゴリズムを用いて $n = 11$ での APN 関数（具体的には $S(x) = x^3$）の Δ-rank を 1 台の PC で計算した例を紹介する．この場合，接続行列全体をメモリに格納すると，それだけで 2^{44} ビット=2TB のメモリが必要となるが，本実装では 2GB 程度で実行でき，計算はおよそ 48 時間で完了した．さらにこのアルゴリズムは接続行列を分割して並列処理させることができるため，マチチスレッド化により複数コア上で実行させたところ，メモリサイズは合計 8.8GB で，25.5 時間で Δ-rank の計算が完了したことを報告する．

なお計算機実験は，特に断りのない限り Intel Core Ultra 7 265, 64GB (DDR5) 搭載の PC で行ったものである．

2. 準備

2.1 S-box と APN 関数，AB 関数

本稿を通じて S を位数 2 の有限体 $\mathbb{F}_2$ 上の n 次元ベクトル空間 $(\mathbb{F}_2)^n$ から自分自身への関数とする．S は S-box をイメージするものであるが全単射であることは仮定しない．また紛れのない限り $(\mathbb{F}_2)^n$ を有限体 $\mathbb{F}_{2^n}$ と同一視する．

S の DDT (differential distribution table) とは式 1 であらわされる二次元テーブル $\mathcal{D}_S(a, b)$ であり，その最大値 $\mathcal{D}(S) := \max_{a\neq 0, b} \mathcal{D}_S(a, b)$ を S の differential uniformity と呼ぶ．$\mathcal{D}_S(a, b)$ が大きいような $a \neq 0$ と b のペアが存在すると，それらの値を差分値と見たときの S の差分確率が高くなるので，differential uniformity が小さいことは差分解読法に対して強い性質ということができる．

© 2026 Information Processing Society of Japan

$$\mathcal{D}_S(a,b) := \#\{x \in (\mathbb{F}_2)^n | S(x) + S(x+a) = b\} \quad (1)$$

$\mathcal{D}(S)$ は S によらず 2 以上の偶数であるが，この値が 2 である時 S は APN (almost perfect nonlinear) であるという．APN 関数は差分解読法に対して最も強い性質をもつことを意味する．例えば $S(x) = x^3$ である時，S はいかなる n に対しても APN 関数となることは容易にわかる（ここでは S を有限体 $\mathbb{F}_{2^n}$ 上の関数とみなしている）が，この関数は n が奇数の時のみ全単射である．また $S(x) = 1/x$ の場合（$S(0) = 0$ と定義する）は，n が奇数の時には APN 関数で，n が偶数の時には $\mathcal{D}(S) = 4$ となることが知られている [12]．n が偶数の場合，全単射であるような APN 関数は $n = 6$ の時に（以下に述べる CCZ 同値を除いて）1 つ知られているだけである [5].

また S の LDT (linear distribution table) とは，式 2 であらわされる二次元テーブル $\mathcal{L}_S(a,b)$ であり，その絶対値の最大値 $\mathcal{L}(S) := \max_{a,b\neq 0} |\mathcal{L}_S(a,b)|$ を linearity と呼ぶ．$|\mathcal{L}_S(a,b)|$ が大きいような a と $b \neq 0$ のペアが存在すると，それらの値をマスク値と見たときの S の線形確率が高くなるので，linearity が小さいことは線形解読法に対して強い性質ということができる．

$$\mathcal{L}_S(a,b) := 2\#\{x \in (\mathbb{F}_2)^n | (a \cdot x) = (b \cdot S(x))\} - 2^n \quad (2)$$

なお $\mathcal{L}_S(a,b)$ は式 3 で示される二次元テーブル $\mathcal{W}_S(a,b)$ と同一であり，この形で用いられることが多い．

$$\mathcal{W}_S(a,b) := \sum_{x \in (\mathbb{F}_2)^n} (-1)^{(a\cdot x)+(b\cdot S(x))} \quad (3)$$

$\mathcal{L}(S)$ は $2^{(n+1)/2}$ 以上であることが知られており，この等号が成り立つ時 S は AB (almost bent) であると呼ばれる．AB 関数は線形解読法に対して最も強い性質をもつことを意味するが，この性質は明らかに n が偶数のときには成り立たない．n が偶数の時の linearity の下限値は $2^{(n/2)+1}$ であろうと予想されているが，未だに証明はされていない．$S(x) = x^3$ の場合，n が奇数なら S は AB 関数となり n が偶数なら $\mathcal{L}(S) = 2^{(n/2)+1}$ となる．また $S(x) = x^{-1}$ の場合，n が偶数ならやはり $\mathcal{L}(S) = 2^{(n/2)+1}$ となる [4].

2.2 S-box の同値関係

差分解読法や線形解読法の文脈で言及される S-box の同値類にはいくつかの種類があり，表 1 に示す 4 つが主な研究対象である．ここで L_i は線形置換，A_i はアフィン置換，B はアフィン関数を示している．また $\{(x, S(x))\}$ とは S のグラフ集合 $\{(x, S(x)) \in (\mathbb{F}_2)^n \times (\mathbb{F}_2)^n | x \in (\mathbb{F}_2)^n\}$ を示している．CCZ 同値は他とは異なり，$2n$ 次元ベクトル集合の同値関係であることに注意する．

線形同値	$S' = L_1 \circ S \circ L_2$
アフィン同値	$S' = A_1 \circ S \circ A_2$
拡大アフィン同値	$S' = A_1 \circ S \circ A_2 + B$
CCZ 同値	$\{(x, S'(x))\} = A_1 \circ \{(x, S(x))\} \circ A_2$

表 1 S と S' の同値関係

これら 4 つの同値関係は表 1 に示された順に包含関係があり，CCZ の同値類が集合として一番大きくなる．このため 2 つの関数が CCZ 同値でなければそれらは「真に異なっている」とみなされる．

一般に CCZ 同値判定問題は符号同値判定問題に帰着されるが，CCZ 同値判定問題の計算量のクラスは知られていない．一般に符号同値判定アルゴリズムを用いて任意の 2 つの関数の CCZ 同値判定を実行するのは重い計算であると考えられている．しかし代数次数が 2 である関数に限定すれば，CCZ 同値は拡大アフィン同値と等価となることが知られており [14]，しかもその同値判定はきわめて簡単に実行できるため [15]，最近の APN 関数探索研究は代数次数 2 のものを対象とするのが主流である．

2.3 CCZ 不変量と Γ-rank, Δ-rank

CCZ 同値である限り値が変わらないような関数の特徴量は，S-box の同値判定の代用あるいは S-box の分類研究に有用である．このような CCZ 不変量はいくつか知られている．例えば differential uniformity や linearity は CCZ 不変量である．また extended Walsh spectrum と呼ばれる，$\mathcal{L}_S(a,b)$（あるいは $\mathcal{W}_S(a,b)$）の絶対値の多重集合（a, b を動かしたときにどの値を何回とるかを示した集合）は CCZ 不変量としてよくとりあげられる．2 つの関数の CCZ 不変量が異なればそれらは CCZ 同値ではないが，一般に逆は成り立たない．したがって CCZ 不変量はその値のバラエティが大きいことが望ましい．

これらよりもはるかに高い解像度をもつ CCZ 不変量が Edel と Pott によって導入された Γ-rank と Δ-rank である．以下ではこれらの定義を説明する．まず $(\mathbb{F}_2)^n \times (\mathbb{F}_2)^n$ を加法群とみなした集合を G と書き，任意の体 $\mathbb{F}$ に対して形式和 $\sum^{\circ}_{g\in G} a_g g$ $(a_g \in \mathbb{F})$ 全体の集合を $\mathbb{F}[G]$ と書く．この集合に以下のように加法・乗法・$\mathbb{F}$ の元の作用を定義すると，$\mathbb{F}[G]$ は群代数 (group algebra) となる．

$$\sum_{g\in G}^{\circ} a_g g + \sum_{g\in G}^{\circ} b_g g := \sum_{g\in G}^{\circ} (a_g + b_g) g \quad (4)$$

$$\sum_{g\in G}^{\circ} a_g g \cdot \sum_{g\in G}^{\circ} b_g g := \sum_{g\in G}^{\circ} \Big(\sum_{h\in G} a_h \cdot b_{g-h} \Big) g \quad (5)$$

$$c\Big(\sum_{g\in G}^{\circ} a_g g \Big) := \sum_{g\in G}^{\circ} (c \cdot a_g) g \quad (6)$$

ここで $\sum_{g\in G}$ が形式和なのに対して，$\sum_{h\in G} a_h \cdot b_{g-h}$ は $\mathbb{F}$ の元の和を示している．これを明示するため形式和は $\sum^{\circ}_{g\in G}$ と記した．

© 2026 Information Processing Society of Japan

与えられた関数 S に対して，この群代数の元 G_S を式 7 のように定義する．これは言い換えると，ある x に対して $g=(x,S(x))$ と書けるなら $a_g=1$，そうでなければ $a_g=0$ とした $\mathbb{F}[G]$ の元であり，S のグラフを表現しているとみなすことができる．

$$G_S := \sum_{x\in(\mathbb{F}_2)^n}^{\circ} (x,S(x)) \tag{7}$$

ここで $\mathbb{F}=\mathbb{F}_2$ の場合に，G_S によって生成される $\mathbb{F}_2[G]$ の単項イデアルの $\mathbb{F}_2$ ベクトル空間としての次元を S の Γ-rank と呼ぶ．Γ-rank とは集合 $\{G_S\cdot(a,b)|(a,b)\in G\}$ で張られるベクトル空間の次元であるので，それは式 8 に示す $2^{2n}\times 2^{2n}$ バイナリ正方行列 $\mathbf{G}_S$ のランクに他ならない．ここで $\mathbf{G}_S[i,j]$ は $\mathbf{G}_S$ の i 行 j 列の値を示している．

$$\mathbf{G}_S[i,j] := \begin{cases} 1 & \text{if } j=(x,S(x))+i \quad {}^{\exists}x\in\mathbb{F}_{2^n} \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

また G_S を $\mathbb{C}[G]$ の元とみなすと，S が APN 関数の時には，係数 a_g がすべて 0 または 1 となる $D_S\in\mathbb{C}[G]$ が存在して式 9 が成り立つ．これは APN 関数の DDT の $(0,0)$ 以外の座標の値が 0 か 2 であることの言い換えである．

$$G_S\cdot G_S = 2^n(0,0)+2D_S \tag{9}$$

この D_S を $\mathbb{F}_2[G]$ の元とみなして，D_S によって生成された $\mathbb{F}_2[G]$ の単項イデアルの $\mathbb{F}_2$ ベクトル空間としての次元を S の Δ-rank と呼ぶ．これは式 10 に示す $2^{2n}\times 2^{2n}$ バイナリ正方行列 $\mathbf{D}_S$ のランクを意味している．

$$\mathbf{D}_S[i,j] := \begin{cases} 1 & \text{if } j=(x_1+x_2,S(x_1)+S(x_2))+i \\ & \quad {}^{\exists}x_1<{}^{\exists}x_2\in\mathbb{F}_{2^n} \\ 0 & \text{otherwise} \end{cases} \tag{10}$$

ここで $\mathbb{F}_{2^n}$ には適当な順序（例えば $\{0,1,2,...,2^n-1\}$ と同一視することで）が定められているものとする．なお G_S は任意の S について定義できるが，D_S は S が APN 関数の時のみ定義されるものであることに注意する．Δ-rank や Γ-rank は CCZ 不変量である．

なお [7] ではデザイン理論の用語を用いて，$(x,S(x))$ を点，$D_S\cdot(a,b)$ あるいは $G_S\cdot(a,b)$ をブロックとするデザインの接続行列 (incidence matrix) として $\mathbf{D}_S$ と $\mathbf{G}_S$ を定義しているが，本稿ではデザイン理論を経由せず，群代数の元 D_S,G_S から直接行列の具体的な定義を与えた．

与えられた S-box から Δ-rank や Γ-rank を求めるにはサイズ $2^{2n}\times 2^{2n}$ のバイナリ行列を扱う必要があり，n のサイズの増加とともにその計算に必要な時間とメモリ量は急速に増大する．次節以降では Δ-rank と Γ-rank の計算手法とその必要リソースを計算機実験を通して考察する．

3. 既存ソフトウエアを用いた Δ-rank と Γ-rank の計算

3.1 Kaleyski らによる実装例

まず，これまでに知られている Δ-rank と Γ-rank の実装例として，Kaleyski による結果 [10] を表 2 に示した．これは商用ソフトウエア Magma を用いて 500GB のメモリをもつサーバーコンピュータ上で実行したもので，著者によればメモリサイズの制約から Δ-rank の計算は $n=9$ まで，Γ-rank は $n=10$ までが計算可能であったとしている．

n	6	7	8	9	10
Δ-rank	2	20	308	6976	-
Γ-rank	2	15	138	4229	899024

表 2 [10] の実装による Rank 計算時間（秒）

Magma は高速な数学計算ソフトウエアとして知られているが，それでも Δ-rank, Γ-rank の計算はかなり重いものであることがわかる．

また同著者らによる最近の論文 [11] では，オープンソースソフトウエア Sagemath をもちいて S-box の様々な指標を計算するソフトウエアフレームワークが提案されており，ここに Δ-rank と Γ-rank の計算時間とメモリ量が示されている．著者らによればこれは高速化を目指したものではないが，メモリ量は極めて大きい実装である．

n	Rank	Time	Memory
7	Δ-rank	1m 23.42sec	2.44GB
	Γ-rank	1m 15.94sec	2.44GB
8	Δ-rank	24m 50.71sec	33.71GB
	Γ-rank	20m 51.72sec	33.53GB
9	Δ-rank	6h 15m 5.00sec	532.45GB
	Γ-rank	5h 57m 15.00sec	530.28GB

表 3 [11] の実装による Rank 計算時間とメモリ量

次に，既存の行列計算ライブラリを用いた，筆者による Δ-rank, Γ-rank 計算プログラムとその結果を示す．

3.2 Sagemath による実装

Δ-rank と Γ-rank の計算は，与えられた関数 S から行列 $\mathbf{D}_S,\mathbf{G}_S$ を生成するステップと，これらの行列のランクを計算するステップからなる．行列生成の計算量は straightforward な方法でもそれぞれ $O(2^{4n}),O(2^{3n})$ であるので，ランク計算よりは高速なはずである．そこで $\mathbf{D}_S,\mathbf{G}_S$ の生成には以下の簡単なアルゴリズムを用いた．ここで記号^と<<はそれぞれ排他的論理和と左シフトを示している．

次のステップに必要な，大きな行列の高速ランク計算には様々な方式が知られているが，大きく分けて疎行列系と密行列系が存在する．前者には Wiedemann アルゴリズムや Lanczos アルゴリズムがあり，これらは古くは素因数分

© 2026 Information Processing Society of Japan

Algorithm 1 $\mathbf{D}_S$ 生成アルゴリズム

Input: n, n-bit S-box S
Output: $2^{2n} \times 2^{2n}$ binary matrix $\mathbf{D}_S$

$\mathbf{D}_S(x, y) = 0$ for all $0 \le x, y < 2^{2n}$
for $x = 0, 1, \ldots, 2^{2n} - 1$ **do**
  for $y1 = 0, 1, \ldots, 2^n - 1$ **do**
    for $y2 = y1{+}1, y1{+}2, \ldots, 2^n - 1$ **do**
      $\mathbf{D}_S[x, ((y1\hat{}y2)\texttt{<<}n)\hat{}(S[y1]\hat{}S[y2])\hat{}x]$ `^= 1`
    end for
  end for
end for

Algorithm 2 $\mathbf{G}_S$ 生成アルゴリズム

Input: n, n-bit S-box S
Output: $2^{2n} \times 2^{2n}$ binary matrix $\mathbf{G}_S$

$\mathbf{G}_S(x, y) = 0$ for all $0 \le x, y < 2^{2n}$
for $x = 0, 1, \ldots, 2^{2n} - 1$ **do**
  for $y = 0, 1, \ldots, 2^n - 1$ **do**
    $\mathbf{G}_S[x, (y\texttt{<<}n)\hat{}S[y]\hat{}x]$ `^= 1`
  end for
end for

解（数体ふるい法）に適用され，現在では多変数多項式暗号など PQC の安全性評価にも登場する．$\mathbf{G}_S$ は各行各列の重みが 2^n であり疎行列に近いともといえるが，これらのアルゴリズムをバイナリ行列の正確なランク計算に利用するにはオーバーヘッドが大きいように思われる．

後者には Four-Russians アルゴリズムや PLE 分解アルゴリズムが知られており，一般には N が小さいと前者が高速，N が大きければ後者が高速とされる．これらのアルゴリズムを C 言語で実装したオープンソースライブラリとして M4RI が知られており，Sagemath にも採用されている．この実装の漸近的計算量は行列サイズ N に対してそれぞれ $O(N^3/\log N)$, $O(N^{\log_2 7}) = O(N^{2.807\ldots})$ である [16]．

Sagemath は数学計算に関するさまざまなオープンソースソフトウエアを統合し，Python から呼び出せる統一的なインターフェースを構築した利便性の高いものであるが，Sagemath で Δ-rank や Γ-rank を計算させる場合，$\mathbf{D}_S$ や $\mathbf{G}_S$ の生成部分を Python で記述するとそこが速度的なボトルネックになる可能性がある（[11] が [10] より低速なのはそれが理由であるように思われる）．そこでこの部分は Cython で記述した次のようなフローのプログラムを作成した．Sagemath では PLE 分解アルゴリズムと Four-Russians アルゴリズムをランク計算時に選択できるので，比較目的で両方を実装した．

```
// 行列宣言
M = matrix(GF(2), 1<<(2*n), 1<<(2*n))
// Cython で記述された接続行列生成関数
create_matrix(M, S, n)
// algorithm は'ple'(デフォルト) か'm4ri'
rank = M.rank(algorithm='ple')
```

このプログラムを $S(x) = x^3$ と $S(x) = x^{-1}$ に関して実行した結果を表 4 と表 5 に示す．n が偶数の場合には $S(x) = x^{-1}$ は APN 関数ではないので Δ-rank は定義されないが，Algorithm 1 は任意の関数 S に対して実行できるものなので，参考までにこのアルゴリズムで生成された行列のランクを Δ-rank として掲載した．

n		Rank	Time(sec)		Memory(MB)	
		$S(x) = x^3$	'ple'	'm4ri'	'ple'	'm4ri'
7	Δ	198	0.34	0.30	520	496
	Γ	3610	0.71	0.89	529	495
8	Δ	420	5.26	5.97	1746	1484
	Γ	11818	16.09	33.04	1636	1484
9	Δ	872	112.22	155.82	21470	17220
	Γ	38470	710.53	2516.59	19080	17220

表 4 Sagemath での $S(x) = x^3$ の Rank 計算時間とメモリ量

n		Rank	Time(sec)		Memory(MB)	
		$S(x) = x^{-1}$	'ple'	'm4ri'	'ple'	'm4ri'
7	Δ	4928	1.25	1.61	532	495
	Γ	8128	1.16	1.36	520	495
8	Δ	21624	31.18	105.06	1651	1484
	Γ	32640	33.38	93.20	1643	1484
9	Δ	93024	1191.32	5648.21	19148	17219
	Γ	130816	1403.15	5135.36	19265	17219

表 5 Sagemath での $S(x) = x^{-1}$ の Rank 計算時間とメモリ量

n が同じでも関数によって（ランクによって）計算時間が大きく異なり，$S(x) = x^3$ はランクが小さい APN 関数，$S(x) = x^{-1}$ はランクが大きい APN 関数の例と見ることができる．また実験の範囲では PLE 分解アルゴリズムが Four-Russians アルゴリズムより高速であることがわかる．[10] や [11] が具体的にどのような関数を計算したのか不明であるが，我々の実装は [10] よりも高速と思われ，またメモリ消費量は [11] よりもはるかに少ないものである．

3.3 M4RI ライブラリによる実装

上記の実装は接続行列の生成に Cython を使っており，また M4RI は C 言語で高度に最適化されているものであるため，実質的には全体が C 言語で書かれているに等しい．M4RI は gcc のライブラリ `libm4ri` としても提供されているため，ここからはオーバーヘッドを最小限にするため Sagemath の統合環境から離れ，Δ-rank, Γ-rank を計算する専用の C 言語プログラムを実装する．

M4RI ライブラリでランク計算を行うには，与えられた行列から階段型 (echelon form) 行列を生成する関数 `mzd_echelonize` を呼び出し，この返り値をランクとして取得するのが普通である．この関数は動的に PLE 分解ア

© 2026 Information Processing Society of Japan

ルゴリズムと Four-Russians アルゴリズムを選択するが，我々の目的では階段型行列は必要はなく，また PLE 分解アルゴリズムが高速なので，これを扱う内部関数 `mzd_ple` を直接呼び出す次のようなフローのプログラムを記述した．

```
// 行列宣言
mzd_t *M = mzd_init(1<<(2*n), 1<<(2*n));
mzp_t *P = mzp_init(1<<(2*n));
mzp_t *Q = mzp_init(1<<(2*n));
// 接続行列生成関数
create_matrix(M, S, n);
// mzd_ple は M を保存しない
rank = (int)mzd_ple(M, P, Q, 0);
// 終了
mzd_free(M);
mzp_free(P);
mzp_free(Q);
```

このプログラムを実行させた結果を表 6 と表 7 に示す．

n	Rank		Time(sec)	Memory(MB)
	$S(x)=x^3$		'ple'	'ple'
7	Δ	198	0.21	56
	Γ	3610	0.71	61
8	Δ	420	2.71	780
	Γ	11818	15.69	643
9	Δ	872	49.59	12326
	Γ	38470	526.48	10015

表 6 M4RI での $S(x)=x^3$ の Rank 計算時間とメモリ量

n	Rank		Time(sec)	Memory(MB)
	$S(x)=x^{-1}$		'ple'	'ple'
7	Δ	4928	1.13	61
	Γ	8128	1.28	54
8	Δ	21624	28.43	650
	Γ	32640	39.06	649
9	Δ	93024	1035.54	10074
	Γ	130816	1594.49	10199

表 7 M4RI での $S(x)=x^{-1}$ の Rank 計算時間とメモリ量

Sagemath のランク計算関数は行列のコピーを内部で作るのに対し，関数 `mzd_ple` は行列 `M` を保存しないのでその分メモリ量が少ない．表の数値はほぼ M4RI ライブラリ内部で動的に確保されるメモリ量に等しく，これは行列を格納するために必要なメモリサイズである 2^{4n} ビットの 25%から 50%増し程度の値である．ここから類推すると，$n=10$ の場合には 160GB–192GB，$n=11$ の場合には 2.5TB–3.0TB のメモリが必要となる．そこで次に，行列のランク計算アルゴリズムを見直すことでメモリ量の削減を試みる．

4. 低ランク行列を考慮した Δ-rank, Γ-rank の計算

4.1 Δ-rank と Γ-rank の値の範囲

これまで Δ-rank, Γ-rank の計算において $\mathbf{D}_S$ や $\mathbf{G}_S$ の構造は特に用いず，任意の $2^{2n}\times 2^{2n}$ 行列のランクを計算するアルゴリズムを用いていた．ここでは $\mathbf{D}_S$ や $\mathbf{G}_S$ のランクが一般には低いことを指摘し，そのことを利用したランク計算アルゴリズムを考えることでメモリ削減を試みる．最初に Δ-rank や Γ-rank はフルランクに比べ最大でも半分にランク落ちすることを示す．

Lemma 1: Δ-rank, Γ-rank は 2^{2n-1} 以下である．

Proof: $\mathbb{F}_2[G]$ から自分自身への $\mathbb{F}_2$ 線形写像 $\sigma(x) : x \mapsto G_S\cdot x$ を考える．この写像の像 $img(\sigma)$ の $\mathbb{F}_2$ 次元が Γ-rank に他ならない．$\mathbb{F}_2[G]$ の標数は 2 であり，また加法群 G も 2 群であるため，σ^2 はゼロ写像となる．したがって $img(\sigma)\subset ker(\sigma)$ である．また $dim(img(\sigma))+dim(ker(\sigma))=|G|=2^{2n}$ であるため，Γ-rank$=dim(img(\sigma))\le 2^{2n-1}$ が示される．Δ-rank の証明も同様である． □

また $\mathbf{G}_S$ の最初の 2^n 個の行ベクトルは明らかに独立であるので Γ-rank は少なくとも 2^n 以上である．これらの上下界はタイトであり，例えば S が定数関数の場合 Γ-rank は 2^n となり，S をランダムな関数とすると多くの場合 Γ-rank は 2^{2n-1} となる．さらに S の代数次数が 2 である時に限れば Δ-rank は次のように非常に小さくなることが知られている（Δ-rank は APN 関数にのみ定義されていることに注意）．$S(x)=x^3$ は典型的な代数次数 2 の関数である．

Lemma 2 ([7])**:** 代数次数が 2 であるような APN 関数の Δ-rank は 2^{n+1} 以下である．

一般に代数次数が高くなると Δ-rank, Γ-rank ともに大きくなる傾向にあり，このことを用いて [7] では代数次数が 2 となる関数と決して CCZ 同値にはならない APN 関数の存在を証明した．

4.2 提案手法

ここではまず汎用的なランク計算手法を述べ，次にランクが一定値以下であることがあらかじめわかっている場合にメモリ量を削減可能な方法に拡張する．Algorithm 3 は，通常の階段形行列を作るアルゴリズムに類似したランク計算方式であるが，pivot 位置を列の下位から 1 ビットずつ決めるのではなく，行ごとに下位ビットから数えて初めて 1 が現れる位置を pivot 位置候補とする（`count_leading_zeros` 関数）点と，列を L ビットごとに分割してこのブロックごとに pivot 位置を決めている点が異なっている．ここで

© 2026 Information Processing Society of Japan

$M[x, y{:}z]$ とは，行列 M の第 x 行のビット位置 y から $z-1$ までの $z-y$ ビットデータを意味している．

Algorithm 3 バイナリ行列 Rank 計算アルゴリズム

Input: n, $2^{2n} \times 2^{2n}$ binary matrix M, $L = 2^\ell$
Output: rank of M

1: **Allocate** $pivot[L]$
2: $rank = 0$
3: **for** $col = 0, L, 2L, \ldots, 2^{2n}-L$ **do**
4: $\quad pivot[y] = NULL$ for all $0 \le y < L$
5: $\quad row = rank$
6: $\quad$ **for** $x = rank, rank{+}1, \ldots, 2^{2n}-1$ **do**
7: $\quad\quad$ **if** $M[x, col{:}col{+}L] = 0$ **continue**
8: $\quad\quad y =$`count_leading_zeros`$(M[x, col{:}col{+}L])$
9: $\quad\quad$ **if** $pivot[y] = NULL$ **then**
10: $\quad\quad\quad$ `swap_row`$(M[x, col{:}2^{2n}], M[row, col{:}2^{2n}])$
11: $\quad\quad\quad pivot[y] = row$
12: $\quad\quad\quad row = row{+}1$
13: $\quad\quad$ **else**
14: $\quad\quad\quad M[x, col{:}2^{2n}]$ `^=` $M[pivot[y], col{:}2^{2n}]$
15: $\quad\quad\quad$ **goto** line 7
16: $\quad\quad$ **end if**
17: $\quad$ **end for**
18: $\quad rank = row$
19: **end for**
20: **Return** $rank$

ソフトウエアでは `count_leading_zeros` 関数は `tzcnt` 命令を使って実現できるので，このアルゴリズムはビットごとにデータを処理する必要がなく，また内部で確保するメモリはブロックごとに pivot 位置に対応する行番号を記憶する L 次元の配列だけですむ．L の値は，プロセッサの 256bit 命令を有効に使うために 256 の倍数が適当であるが，実験では $L = 256$ の時が最も高速であったため $L = 256$ での実験結果を表 8 と表 9 に示した．

n	Rank		Time(sec)	Memory(MB)
	$S(x) = x^3$		Algorithm 3	Algorithm 3
7	Δ	198	0.11	36
	Γ	3610	0.46	36
8	Δ	420	2.52	516
	Γ	11818	16.13	516
9	Δ	872	80.37	8196
	Γ	38470	1927.68	8196

表 8 提案手法での $S(x) = x^3$ の Rank 計算時間とメモリ量

このアルゴリズムは C 言語で 50 行程度で記述できる簡単なものであり，計算量も漸近的に高速なものではないが，それでも全体に M4RI ライブラリに実装された Four-Russians アルゴリズムより高速である．PLE 分解アルゴリズムと比較すると $n = 8$ では同程度だが，$n = 9$ になると PLE 分解アルゴリズムの方が高速となる．

一方で，このアルゴリズムは接続行列生成とランク計算をインターリーブさせることができる，すなわち接続行列を少しづつ生成しながらランク計算を実行できるため，このランクの上限値 `max_rank` があらかじめわかっている場合には行列すべてをメモリに格納する必要はなく，(`max_rank + const`) $\times\ 2^{2n}$ ビット程度のメモリさえ確保すれば，速度を落とすことなくランク計算が可能となる．具体的にこれを Δ-rank 計算の場合に書き下したアルゴリズムを Algorithm 4 に示す．Γ-rank の場合も同様である．B は一度に生成する接続行列の行数を示している．なおこのアルゴリズムでは一度得られた pivot 位置情報をすべて持ち続ける必要があるので，その配列サイズは 2^{2n} となる．

n	Rank		Time(sec)	Memory(MB)
	$S(x) = x^{-1}$		Algorithm 3	Algorithm 3
7	Δ	4928	0.51	36
	Γ	8128	0.65	36
8	Δ	21624	29.79	516
	Γ	32640	39.38	516
9	Δ	93024	2657.52	8196
	Γ	130816	4005.45	8196

表 9 提案手法での $S(x) = x^{-1}$ の Rank 計算時間とメモリ量

4.3 実験結果

このアルゴリズムを用いて $S(x) = x^3$ の場合に Δ-rank を $n = 9$ から $n = 11$ まで計算した結果を表 10 に示す．Lemma 2 より max_row は $2^{n+1} + B$ とすれば十分である．実験では B の値は 256 としている．

n	Rank		Time(sec)	Memory(MB)
	$S(x) = x^3$		Algorithm 4	Algorithm 4
9	Δ	872	67.81	42
10	Δ	1804	3855.95	270
11	Δ	3698	173644.93	2013

表 10 インターリーブによる Δ-rank 計算時間とメモリ量

Δ-rank が小さい分メモリ量が少なくなっており，$n = 11$ でも 2GB 程度で計算できることがわかる．

さらにこの計算をマルチスレッド化によって高速化することを試みた．具体的には $\mathbf{D}_S$ を 4 つの $2^{2n-2} \times 2^{2n}$ 行列に分離し，それぞれを Algorithm 4 で並列計算させ，得られた 4 つの pivot 行列を順に積み重ねながら再び Algorithm 4 を用いて Δ-rank の計算をさせるものである．並列計算の結果得られた 4 つの pivot 行列のサイズはすでに Δ-rank に近いものであることが想定されるため，後半の計算時間は前半の計算時間に比べて無視できると考えられる．これを $n = 9$ から $n = 11$ まで実装した結果を表 11 に示す．

期待としてはメモリ量が 4 倍になる代償として速度も 4 倍になることであったが，$n = 9$ においては概ねそれに近い結果が得られたものの，$n = 10, 11$ では速度向上は 2 倍程度にとどまった．4 つのスレッドは論理的には独立でメモリ上のデータの競合もおきないものだが，ランク計算は

© 2026 Information Processing Society of Japan

Algorithm 4 Δ-rank インターリーブ計算アルゴリズム

Input: n, S-box S, $max_rank, L = 2^{\ell}, B = 2^{b}$
Output: Δ-rank of S, or Exceed_max_rank
// 初期化
1: **Allocate** $pivot[2^{2n}]$
2: **Allocate** $M[max_rank{+}B, 2^{2n}]$
3: $pivot[y] = NULL$ for all $0 \le y < 2^{2n}$
4: $M[x, y] = 0$ for all $0 \le x < max_rank{+}B$ and $0 \le y < 2^{2n}$
// メインループ（z は $\mathbf{D}_S$ の行，x は M の行)
5: $rank = 0$
6: $z = 0$
7: **while** $z < 2^{2n}$ **do**
8: **if** $rank > max_rank$ **then**
9: **Return** Exceed_max_rank
10: **end if**
// $\mathbf{D}_S$ を B 行だけ生成し M に追加
11: **for** $i = 0, 1, \ldots, B-1$ **do**
12: **for** $y1 = 0, 1, \ldots, 2^n-1$ **do**
13: **for** $y2 = y1{+}1, y1{+}2, \ldots, 2^n-1$ **do**
14: $M[rank{+}i][((y1\hat{}y2)<<n)\hat{}(S[y1]\hat{}[y2])\hat{}z]\hat{}{=}1$
15: **end for**
16: **end for**
17: $z = z{+}1$
18: **end for**
// M の Rank 計算（追加した行から再開する）
19: $max_x = rank{+}B$
20: **for** $col = 0, L, 2L, \ldots, 2^{2n}-L$ **do**
21: $row = rank$
22: **for** $x = rank, rank{+}1, \ldots, max_x{-}1$ **do**
23: **if** $M[x, col{:}col{+}L] = 0$ **continue**
24: $y =$`count_leading_zeros`$(M[x, col{:}col{+}L])$
25: **if** $pivot[y{+}col] = NULL$ **then**
26: `swap_row`$(M[x, col{:}2^{2n}], M[row, col{:}2^{2n}])$
27: $pivot[y{+}col] = row$
28: $row = row{+}1$
29: **else**
30: $M[x, col{:}2^{2n}]$ `^=` $M[pivot[y{+}col], col{:}2^{2n}]$
31: **goto** line 23
32: **end if**
33: **end for**
34: $rank = row$
35: **end for**
36: **end while**
37: **Return** $rank$

メモリアクセスが頻繁なアルゴリズムであるため，PC のメモリバストラフィックがボトルネックになっていると考えられる．実際このプログラムを別のサーバー計算機で実行したところ $n = 11$ でも 4 倍に近い性能が得られている（性能数値の詳細は紙面の都合で割愛した)．

n	Rank		Time(sec)	Memory(MB)
	$S(x) = x^3$		Algorithm 4	Algorithm 4
9	Δ	872	21.32	173
10	Δ	1804	2001.24	1190
11	Δ	3698	91996.58	8838

表 11 マルチスレッドによる Δ-rank 計算時間とメモリ量

5. おわりに

本稿では S-box の接続行列のランクが低いことに注目し，その Γ-rank と Δ-rank の計算手法を主に必要メモリの削減の観点から考察し計算機実験で実証した．提案アルゴリズムはランクが低いほど効果が高いので，次の疑問は APN あるいはそれに類する関数のランクの分布である．既知の APN 関数の中では $S(x) = x^3$ はランクが最も低く，$S(x) = x^{-1}$ はランクが最も高いものであるが，多くの APN 関数のランクは x^3 側，つまり低い方に寄っているように思われる．これについては別稿にて考察をおこなう．

参考文献

[1] Biham, E., Shamir, A.: *Differential Cryptanalysis of DES-like Cryptosystems*, CRYPTO'90 (1990).
[2] Matsui, M.: *Linear Cryptanalysis Method for DES Cipher*, Eurocrypt'93 (1993).
[3] Nyberg, K., Knudsen, L.: *Provable Security Against Differential Cryptanalysis*, CRYPTO'92 (1992).
[4] Chabaud, F., Vaudenay, S.; *Links between Differential and Linear Cryptanalysis*, Eurocrypt'94 (1994).
[5] Browning, K.A., Dillon, J.F., McQuistan, M.T., Wolfe, A.J.: *An APN permutation in dimension six*, 9th International Conference on Finite Fields and Applications (2009).
[6] Beierle, C., Langevin, P., Leander, G., Polujan, A., Rasoolzadeh, S.: *Millions of inequivalent quadratic APN functions in eight variables*, 10th Internatinal Workshop on Boolean Functions and their Applications (2025).
[7] Edel, Y., Pott., A.: *A new almost perfect nonlinear function which is not quadratic*, Advances in Mathematics of Communications, 2009, 3(1): 59-81 (2009).
[8] Carlet, C., Charpin, P., Zinoviev, V.: *Codes, Bent Functions and Permutations Suitable For DES-like Cryptosystems*, Designs, Codes and Cryptography, 15, 125-156 (1998).
[9] Petrank, E., Roth, R.M.: *Is Code Equivalence Easy to Decide?*, IEEE Transactions on Information Theory, 43(5), 1602-1604 (1997).
[10] Kaleyski, N.: *Invariants for EA- and CCZ-equivalence of APN and AB functions*, Cryptography and Communications, Volume 13, Issue 6, 995-1023 (2021).
[11] Kaleyski, N., Matre, A.: *A unified software framework for classification of vectorial Boolean functions*, 10th Internatinal Workshop on Boolean Functions and their Applications (2025).
[12] Nyberg, K.; *Differentially uniform mappings for cryptography*, Eurocrypt'93 (1993).
[13] Canteaut, A., Perrin, L.: *On CCZ-equivalence, extended-affine equivalence, and function twisting*, Finite Fields and Their Applications 56 209-246 (2019).
[14] Yoshiara, S.: *Equivalences of power apn functions with power or quadratic apn functions*, Journal of Algebraic Combinatorics 44(3) 561-585 (2016).
[15] Canteaut, A., Couvreur, A., Perrin, L.: *Recovering or Testing Extended-Affine Equivalence*, IEEE Transactions on Information Theory, Volume 68, Issue 9, 6187-6206 (2022).
[16] Albrecht, M., Bard, G.: *The M4RI Library*, The M4RI Team. `https://bitbucket.org/malb/m4ri`

© 2026 Information Processing Society of Japan